

Bản trình chiếu: Tư tưởng nhân đôi trong thi tin học

Zhu Chenguang

2005

Nguồn gốc: Bàn sơ về ứng dụng của tư tưởng nhân đôi trong thi tin học: bản trình chiếu

IOI China candidate team papers / 2005 / Zhu Chenguang / Zhu Chenguang.ppt

1 Mạch chính của slide

Bản trình chiếu đi cùng bài viết *Bàn sơ về ứng dụng của tư tưởng nhân đôi trong thi tin học*. Phần chữ trích xuất được từ PowerPoint cũ gồm các mốc chính: tính a^n bằng biểu diễn nhị phân của n , ví dụ xức xắc với $24 \cdot 4 = 96$ trạng thái mỗi cột và Dijkstra, khuôn “Doubling Algorithm”, bảng thưa $B[i, j]$, truy vấn RMQ trong $\mathcal{O}(1)$, bảng nhảy cho LCA, và phần thảo luận vì sao chọn mở rộng gấp đôi thay vì gấp k . Bản dịch này giữ các ý thuật toán của slide nhưng không chép lại khuôn mã giả dài hoặc các chương trình phụ lục.

Thông điệp trung tâm là: khi cùng một quy luật biến đổi được áp dụng nhiều lần, hoặc khi thông tin trên đoạn có thể ghép từ hai nửa, ta có thể tính trước các khối độ dài 1, 2, 4, Sau đó mọi số bước hoặc mọi đoạn cần xét được phân rã theo các lũy thừa của 2.

2 Từ lũy thừa nhanh tới quy luật biến đổi

Ví dụ mở đầu là tính a^n . Cách tuần tự cần $n - 1$ phép nhân. Cách nhân đôi tính lần lượt

$$a, a^2, a^4, a^8, \dots$$

bằng phép bình phương. Nếu

$$n = 2^{b_1} + 2^{b_2} + \dots + 2^{b_w},$$

thì

$$a^n = a^{2^{b_1}} a^{2^{b_2}} \dots a^{2^{b_w}}.$$

Số phần tử cần dùng không vượt quá số bit của n , nên thời gian giảm từ $\mathcal{O}(n)$ xuống $\mathcal{O}(\log n)$.

Slide nhấn mạnh rằng đối tượng nhân đôi không nhất thiết là số. Nó có thể là một ma trận, một phép chuyển trạng thái, hoặc một quan hệ “đi từ trạng thái này tới trạng thái kia sau k bước”. Điều kiện cần là quy luật biến đổi không đổi theo thời gian hoặc theo vị trí, và phép ghép hai quy luật có tính kết hợp. Khi đó ta có thể tính quy luật cho 2 bước từ quy luật cho 1 bước, quy luật cho 4 bước từ quy luật cho 2 bước, và tiếp tục như vậy.

3 Ví dụ xúc xắc và min-plus

Ví dụ trong slide là bài lăn xúc xắc trên một dải bốn hàng kéo dài vô hạn theo chiều ngang. Mỗi trạng thái gồm vị trí hàng và hướng của xúc xắc; vì xúc xắc có 24 hướng và có 4 hàng, mỗi cột có 96 trạng thái. Nếu khoảng cách theo cột nhỏ, ta có thể coi các cột là các lớp và dùng quy hoạch động hoặc Dijkstra giữa hai cột kề nhau. Nhưng tọa độ cột có thể rất lớn, nên đi từng cột là không đủ.

Quan sát để dùng nhân đôi là chi phí chuyển từ cột i sang cột $i + k$ không phụ thuộc vào i , chỉ phụ thuộc vào k và hai trạng thái đầu cuối. Trước hết tính ma trận chuyển M_1 , trong đó $M_1[A, B]$ là chi phí nhỏ nhất để đi từ trạng thái A ở một cột tới trạng thái B ở cột kế tiếp. Từ đó ghép ma trận để có $M_2, M_4, M_8, \dots$

Phép ghép ở đây không phải nhân ma trận thường, mà là phép min-plus:

$$(X \circ Y)[i, j] = \min_k \{X[i, k] + Y[k, j]\}.$$

Nếu độ lệch cột là n , ta phân rã n theo nhị phân và lần lượt áp dụng các ma trận tương ứng cho vector trạng thái ban đầu. Slide ghi độ phức tạp xấp xỉ $\mathcal{O}(96^3 \log N)$. Con số 96 là hằng số của bài, còn ý tưởng tổng quát là nhân đôi trên chính quy luật chuyển trạng thái.

4 Bảng thưa cho thao tác đoạn

Hướng ứng dụng thứ hai là tiền xử lý thông tin trên đoạn. Với mỗi vị trí i và mỗi j , bảng thưa lưu thông tin của đoạn

$$[i, i + 2^j - 1].$$

Mục $j + 1$ được ghép từ hai mục j :

$$[i, i + 2^j - 1] \quad \text{và} \quad [i + 2^j, i + 2^{j+1} - 1].$$

Tiền xử lý mất $\mathcal{O}(N \log N)$.

Trong RMQ, $B[i, j]$ lưu chỉ số phần tử nhỏ nhất trên đoạn bắt đầu tại i có độ dài 2^j . Với truy vấn $[i, j]$, đặt

$$k = \lfloor \log_2(j - i + 1) \rfloor.$$

Đoạn truy vấn được phủ bởi hai đoạn độ dài 2^k :

$$[i, i + 2^k - 1], \quad [j - 2^k + 1, j],$$

hai đoạn này có thể chồng lên nhau. Vì phép lấy min không bị ảnh hưởng bởi chồng lấn, chỉ cần so sánh hai ứng viên trong bảng, thời gian truy vấn là $\mathcal{O}(1)$. Với các phép không cho phép chồng lấn, ta vẫn phân rã đoạn thành các khối lũy thừa của hai và trả lời trong $\mathcal{O}(\log N)$.

5 Bảng nhảy trên cây

Slide cũng nhắc LCA như một ví dụ đoạn không hiện ra trực tiếp trong đề. Với mỗi đỉnh v , lưu cha của v , tổ tiên cách 2 cạnh, cách 4 cạnh, v.v. Bảng này được điền bằng công thức

$$up[v][j + 1] = up[up[v][j]][j].$$

Khi cần tìm tổ tiên chung gần nhất của u, v , trước hết nâng đỉnh sâu hơn lên cùng độ sâu bằng các bước nhảy lũy thừa hai; sau đó cùng nâng cả hai từ bước lớn xuống nhỏ cho tới trước LCA. Mỗi truy vấn mất $\mathcal{O}(\log N)$.

Điểm slide muốn giữ lại là nhân đôi không chỉ áp dụng cho mảng. Trên cây, “đoạn” là đường tổ tiên do ta tự tạo ra; bảng nhảy chỉ là một cách lưu các đoạn tổ tiên có độ dài lũy thừa của hai.

6 Vì sao là nhân đôi

Phần cuối đặt câu hỏi: nếu đã có thể mở rộng gấp đôi, có thể mở rộng gấp ba hoặc gấp k không? Về nguyên tắc là có. Nhưng với cơ số k , số tầng là $\log_k N$, còn để ghép một khối lớn gấp k lần thường cần $k - 1$ phép ghép. Tổng chi phí tỷ lệ với

$$(k - 1) \log_k N.$$

So với $\log_2 N$, tỉ số là

$$\frac{k - 1}{\log_2 k}.$$

Với $k = 2$ tỉ số bằng 1, còn với $k \geq 3$ thì $k - 1 > \log_2 k$. Vì vậy gấp đôi là lựa chọn tự nhiên: vừa có số tầng logarit, vừa tốn ít phép ghép nhất để đi từ tầng này sang tầng kế tiếp.

7 Kết luận

Nhân đôi là cách tận dụng lại kết quả đã tính ở mức rất cao. Trong chuyển trạng thái, ta nhân đôi quy luật; trong thao tác đoạn, ta nhân đôi độ dài đoạn đã biết; trong cây, ta nhân đôi bước nhảy tổ tiên. Khi bài toán có tính kết hợp và nhiều thao tác lặp, đây thường là con đường ngắn từ lời giải tuyến tính hoặc bậc cao xuống lời giải logarit.